

Midnight Buildathon · Wave 1

One pledge per invoice. No lender sees another's book.

A registry on Midnight that refuses to let the same receivable be financed twice, while each lender sees only its own deals.

Try it: `onepledge.vercel.app`

`github.com/0oJae/onepledge`

The problem

The same invoice, pledged to two lenders. Both pay out.

Lenders can't compare notes

Catching a double pledge means pooling books. A lender's clients and pricing are its business, so nobody shares.

Hash registry: bypassed

FV/2026/09/0412 and FV-2026-09-0412 hash differently. Each bank's spelling is a "new" invoice.

Hash registry: leaks

Anyone holding an invoice hashes it and learns from the public chain whether, and when, it was financed.

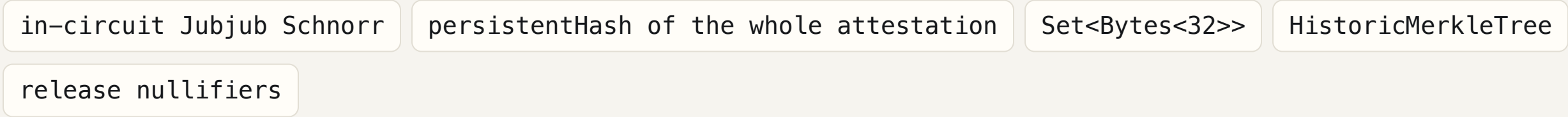
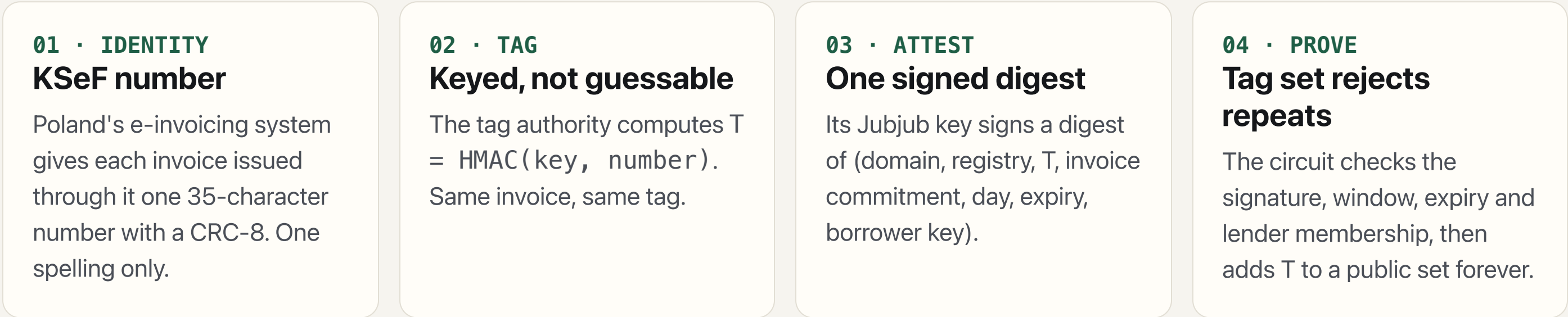
Keep the check public and the books private.

	OPERATOR-RUN REGISTRY	PUBLIC HASH REGISTRY	ONEPLEDGE ON MIDNIGHT
Who sees each lender's deals	The operator	Anyone who can hash the invoice	Nobody
Dodged by a spelling change	No	Yes	No: one KSeF number, keyed tag
Outsiders can see who financed what	No	Yes	No: tags need the authority's key
Rule checked by code anyone can verify	No	Yes	Yes: in-circuit, verifier keys on chain
Still trusted	The operator, for everything	Nothing, but it leaks	The tag authority, to tag each invoice once; the 1-of-1 upgrade key

Midnight gives private state with public verification: attestations, secrets and lender identity stay local; only what a circuit discloses reaches the ledger.

How it works

A canonical ID, a keyed tag, and a proof checked on Midnight.



Selective disclosure

Everyone learns what they need, and no more.

PARTY	LEARNS	NEVER LEARNS
Financing lender	full invoice checked against the signed commitment	Other lenders' deals
Rival lender given a tag	"already pledged"	By whom, how much, which terms
Public chain	Opaque tags, note commitments, roots, expiry days, counts	invoice, amount, borrower, lender
Tag authority	Its attested invoices, and whether and when each was pledged	Which lender financed them, or the terms

Enforced by privacy tests that check every value longer than two bytes in each circuit's public transcript against an allowlist.

Deployed, exercised, and checkable without a wallet.

CONTRACT	567df565...0c2355
DEPLOYED	block 2550929
SCENARIO	2 lenders admitted, 2 pledges, 1 release; the double pledge refused before proving
PROVING TIME	0.9–4.5 s per transaction on a laptop
UPGRADE KEY	1-of-1 deployer key, disclosed

`npm run verify:onchain` compares every on-chain verifier key with this repository.

1 OnePledge

[The story](#) [Why not a hash registry](#) [Live registry](#)

Live on Midnight Preprod · registry v2 · [read from the public indexer in your browser](#)

What the chain actually holds.

No wallet and no OnePledge server. Your browser asks the public indexer for the contract state and decodes it with the contract's own compiled ledger reader. The values below are as reported by that indexer.

CONTRACT	DEPLOYED	ACCEPTS INVOICES	UPGRADE KEY
567df56556...0c2355	block 2550929	2026-07-01 to 2027-01-01	1-of-1 deployer key

Read again

State read at 20:45:11 UTC, latest contract action in block 2550985.

Counters

PLEDGES2

RELEASES1

ADMITTED LENDERS2

PLEDGE NOTES2

Tags (every receivable ever pledged)

cdcbbf13534ca1aad382029efc3244fe0344de781c7b7f6319755119da228aa3e75b2cf84441e5c94ceafb78283ddaf74a2db6a4cfb9ab298ec7c17c8aa468ce

Opaque keyed hashes of KSeF numbers. No invoice number, amount or borrower appears in state; lenders appear only as admitted-lender key commitments, not linked to any pledge.

Trust anchors

TAG AUTHORITY KEY6bedeef4ce...c16845

WINDOW (FROM STATE)2026-07-01 to 2027-01-01

We attacked it before anyone else could.

176

tests: adversarial witnesses, forgery, privacy allowlists

88

findings from our own adversarial self-review, each challenged by a skeptic

1

critical bug found in v1, fixed in v2, disclosed in full

F01: one attestation, 256 pledges

v1 signed only 248 bits of the tag, so changing its last byte passed. v2 signs one SHA-256 digest of the whole attestation, bound to the registry and an expiry. A regression test flips every byte.

Honest limits

- Single tag authority (threshold in Wave 3)
- No lender consent step or void path yet (Wave 2)
- Synthetic KSeF numbers until the test-environment adapter

Start where every invoice already has one ID.

Beachhead: factoring on KSeF

Poland's national e-invoicing gives B2B invoices a canonical number. Buyers: factors, supply-chain-finance banks, private-credit funds. Distribution: the e-invoicing software that already connects them.

Useful with just two lenders. No consortium needed.

Model: a KSeF-connected e-invoicing provider or lenders' association runs the tag authority; lenders pay per attestation checked or pledge registered.

Next

- **Wave 2:** Lace-connected borrower desk and lender inbox, lender acceptance and void path, encrypted invoice delivery, KSeF test-environment adapter, 2-of-3 upgrade committee, practitioner interviews
- **Wave 3:** threshold tags, pledge assignment between lenders, a reusable uniqueness module for any guessable-ID registry

OoJae, solo builder.

Built in Wave 1

- Compact registry contract and in-circuit Schnorr
- Tag authority, CLI and Preprod deployment
- Web demo, security review and threat model

Find it

DEMO	onepledge.vercel.app
CODE	github.com/OoJae/onepledge
GITHUB	@OoJae